Conceptos esenciales

El Software: Proceso y Producto

En la ingeniería de software, es fundamental distinguir entre lo que se construye y cómo se construye. Ambos enfoques son interdependientes; un proceso deficiente rara vez produce un producto de alta calidad.

- Software como Producto: Es el resultado tangible final que se entrega al usuario o cliente. No se limita únicamente al código fuente y ejecutable, sino que incluye toda la información asociada: bases de datos, documentación técnica, manuales de usuario y configuraciones. El foco está en el qué se entrega.

- Software como Proceso: Es el marco de trabajo estructurado de actividades, métodos, herramientas y prácticas utilizadas para desarrollar y mantener el producto. Incluye fases como especificación de requerimientos, diseño, implementación, pruebas y despliegue. El foco está en el cómo se construye.

Ciclos de Vida del Software

Un ciclo de vida de desarrollo de software es un modelo conceptual que describe las fases por las que pasa un proyecto de software desde su concepción inicial hasta su retiro. Se dividen principalmente en dos paradigmas: tradicionales y adaptativos.

Ciclos de Vida Tradicionales

Se basan en un enfoque secuencial y planificado. Asumen que los requerimientos pueden definirse con precisión desde el inicio del proyecto y que el entorno no sufrirá cambios significativos.

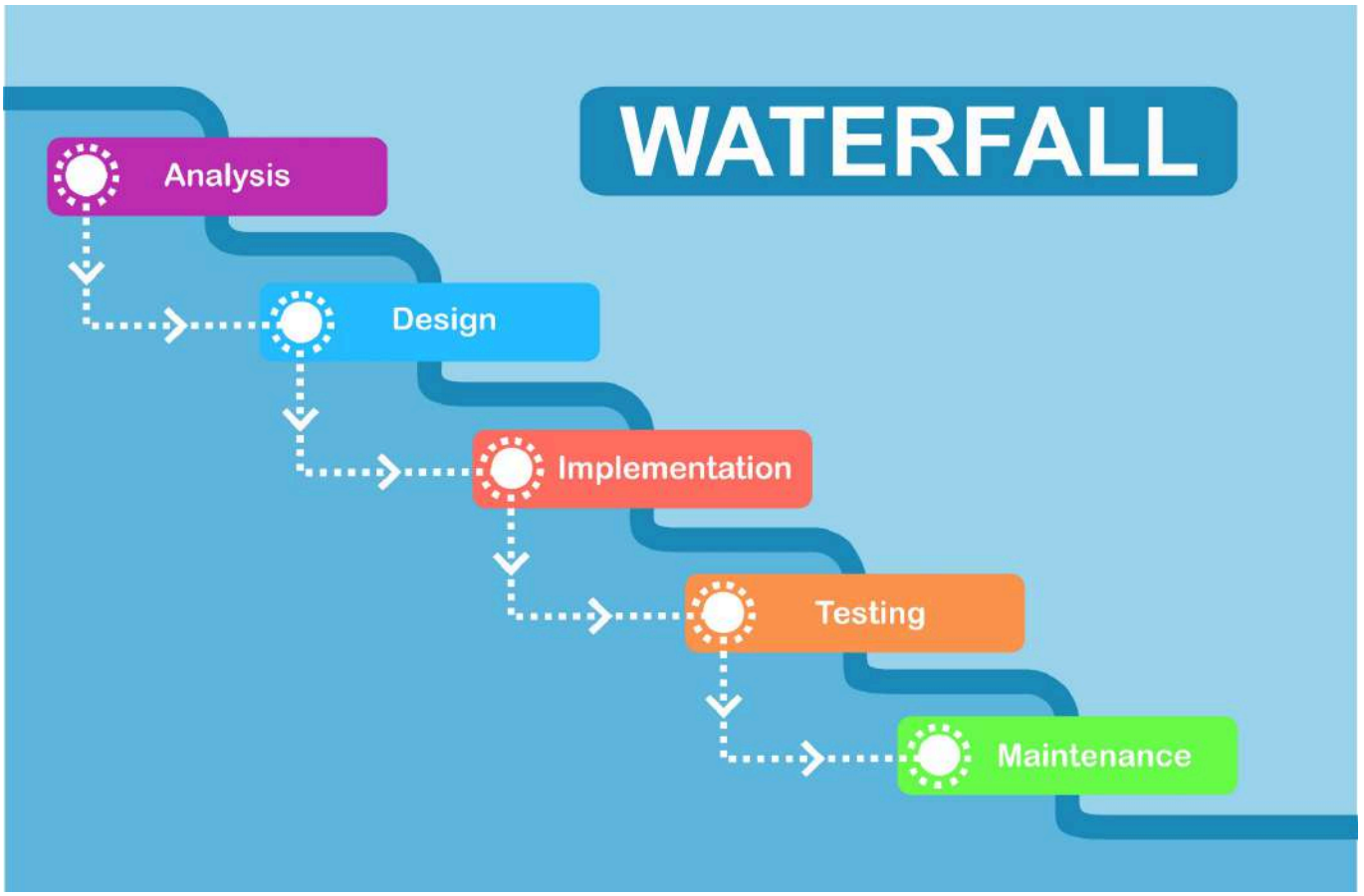
Características principales:

- Fases estrictamente secuenciales (ej. Análisis > Diseño > Codificación > Pruebas > Mantenimiento).

- Cada fase debe completarse y documentarse antes de pasar a la siguiente.

- Alta predictibilidad en costos y tiempos (si los requerimientos no cambian).

Modelo representativo: Modelo en Cascada (Waterfall).



- Ventajas: Fácil de gestionar, hitos bien definidos, documentación exhaustiva.
- Desventajas: Alta rigidez. Dificultad y alto costo para incorporar cambios en fases avanzadas. El cliente no ve software funcional hasta el final del ciclo.

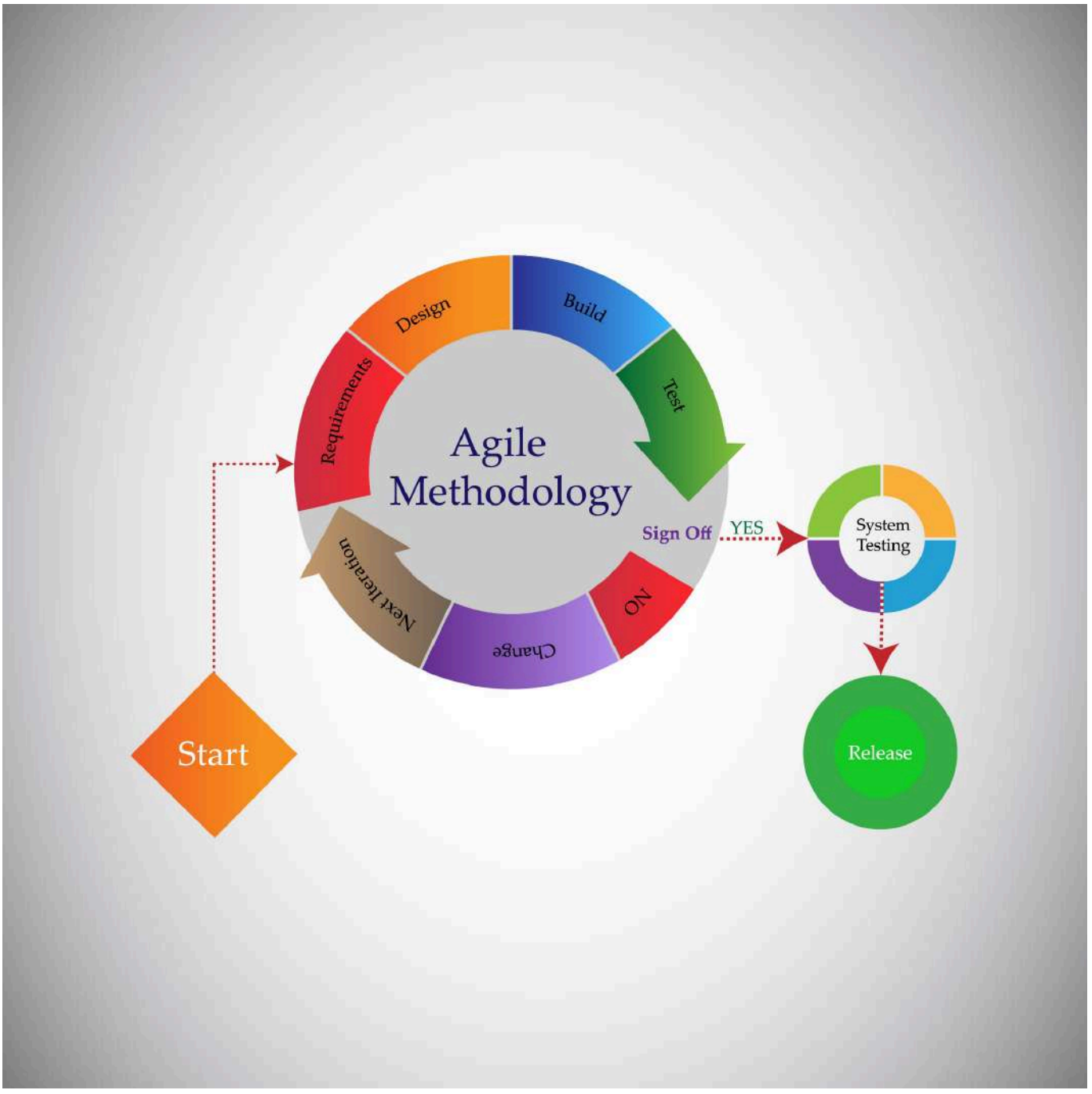
Ciclos de Vida Adaptativos

Se basan en un enfoque iterativo e incremental. Asumen que los requerimientos son volátiles y que la adaptación continua es necesaria para entregar valor.

Características principales:

- El desarrollo se divide en ciclos cortos (iteraciones o sprints).
- Al final de cada iteración, se entrega un incremento de software funcional y evaluable.
- Los requerimientos se priorizan y refinan continuamente.

Modelo representativo: Scrum, Kanban, Extreme Programming (XP).



- Ventajas: Alta flexibilidad ante cambios, mitigación temprana de riesgos técnicos, retroalimentación continua del cliente, entregas rápidas de valor.

- Desventajas: Dificultad para predecir el costo y tiempo final total, requiere alta disponibilidad y compromiso del cliente, riesgo de documentación insuficiente.

Ingeniería de Requerimientos

La Ingeniería de Requerimientos es el proceso sistemático de descubrir, documentar y mantener los requisitos de un sistema de software. Su objetivo principal es establecer un entendimiento común entre el cliente y el equipo de desarrollo sobre lo que se debe construir, minimizando riesgos y retrabajos en etapas avanzadas.

Tipos de Requisitos

Es fundamental clasificar los requerimientos para asegurar tanto la funcionalidad como la calidad del sistema.

- Requisitos Funcionales: Describen qué debe hacer el sistema. Definen las funciones, servicios, cálculos o procesamiento de datos que el software debe ejecutar como respuesta a entradas específicas.
    - Ejemplo: "El sistema debe permitir al alumno descargar el certificado analítico en formato PDF."
- Requisitos No Funcionales: Describen cómo debe comportarse el sistema. Establecen restricciones sobre los servicios o funciones ofrecidos, abarcando atributos de calidad como rendimiento, seguridad, usabilidad, escalabilidad y fiabilidad.
    - Ejemplo: "La generación y descarga del certificado analítico en PDF no debe demorar más de 3 segundos."
- Requisitos de Dominio: Surgen del área de aplicación del sistema (educación, finanzas, salud) y no de las necesidades directas del usuario. Representan leyes o normativas propias del sector que el software debe cumplir obligatoriamente.
    - Ejemplo: "El cálculo del promedio académico debe procesarse utilizando la escala de calificaciones del 1 al 10 y excluir las materias extracurriculares, conforme al reglamento universitario vigente."

El Proceso de Ingeniería de Requerimientos

El desarrollo de requerimientos no es una actividad de un solo paso, sino un proceso iterativo compuesto por cuatro fases principales.

1. Elicitación (Descubrimiento)

Es la fase de extracción y recopilación de las necesidades de los stakeholders (clientes, usuarios finales, expertos del dominio). El mayor desafío es que los usuarios a menudo no saben exactamente lo que quieren o no saben cómo expresarlo técnicamente.

- Técnicas comunes: Entrevistas estructuradas o abiertas, cuestionarios, talleres (workshops), observación directa del entorno de trabajo, análisis de sistemas existentes o documentación legal.

2. Análisis

Consiste en procesar la información obtenida en la elicitación para clasificar, organizar y priorizar los requisitos. En esta fase se detectan y resuelven conflictos (por ejemplo, cuando dos áreas de la empresa piden funcionalidades contradictorias) y se evalúa la viabilidad técnica y económica.

- Actividades: Modelado conceptual (diagramas de flujo, casos de uso), negociación con stakeholders, identificación de riesgos.

3. Especificación

Es la redacción y formalización de los requerimientos analizados en un documento o formato estructurado. Este artefacto debe ser claro, consistente, completo y libre de ambigüedades, ya que servirá como contrato y guía para el desarrollo y las pruebas.

- Formatos comunes: Documento de Especificación de Requerimientos de Software (SRS - IEEE 830), Historias de Usuario (en metodologías ágiles), especificaciones de Casos de Uso.

4. Validación

Es el proceso de revisión final para confirmar que los requerimientos especificados representan exactamente lo que el cliente necesita. Su objetivo es responder a la pregunta: "¿Estamos construyendo el producto correcto?". Encontrar un error aquí es significativamente más barato que encontrarlo durante la programación o después de la entrega.

- Técnicas comunes: Revisiones formales (inspecciones del documento), creación de prototipos de interfaz (mockups) para validación visual con el usuario, generación de casos de prueba preliminares..

Gestión de requerimientos asistida por IA

La integración de Modelos de Lenguaje Grande (LLMs) en la Ingeniería de Requerimientos permite automatizar tareas operativas, reducir errores de interpretación y estandarizar la documentación. La eficacia de estas herramientas depende directamente de la aplicación de técnicas de Prompt Engineering (Ingeniería de Instrucciones).

Técnicas de Prompt Engineering en Requerimientos

Para obtener resultados precisos y útiles en el contexto del desarrollo de software, un prompt (instrucción) debe estructurarse utilizando parámetros claros. Un estándar efectivo incluye cuatro componentes:

1. Rol: Define la perspectiva técnica de la IA (ej. "Actúa como un Analista Funcional Senior").

2. Contexto: Establece las reglas de negocio y el entorno del sistema.

3. Tarea: Especifica la acción exacta a realizar.

4. Formato: Determina la estructura de salida requerida (ej. formato tabular, Gherkin, plantilla de Casos de Uso).

Refinamiento de Historias de Usuario

Las Historias de Usuario iniciales suelen ser esquemáticas. La IA se utiliza para expandir estas descripciones, asegurar que cumplan con criterios de calidad (como el acrónimo INVEST: Independiente, Negociable, Valiosa, Estimable, Pequeña, Testeable) y redactar los Criterios de Aceptación.

Elemento	Ejemplo Práctico
Prompt de Entrada	"Actúa como Product Owner. Toma esta historia de usuario preliminar: *'Como usuario quiero recuperar mi contraseña para volver a entrar al sistema'*. Reescríbela siguiendo el estándar INVEST y genera un de aceptación en formato BDD (Given-When-Then)."

Salida Generada (IA)	**HU Refinada:** Como usuario registrado, quiero restablecer mi contraseña mediante un enlace enviado a mi correo electrónico, para recuperar el acceso a mi cuenta de forma segura. **Criterios de Aceptación:** *Given* que el usuario ingresa un email válido, *When* hace clic en "Recuperar", *Then* el sistema envía un enlace temporal válido por 24 horas.

Detección de Ambigüedades

Los requerimientos redactados en lenguaje natural son propensos a la vaguedad y a contradicciones lógicas. La IA actúa como un revisor sintáctico y semántico, identificando términos subjetivos (ej. "rápido", "intuitivo", "seguro") y requiriendo su cuantificación u objetivación.

Elemento	Ejemplo Práctico
Prompt de Entrada	"Actúa como Auditor de Calidad de Software. Analiza el siguiente requerimiento: *'El sistema debe procesar los pagos de forma rápida y la interfaz debe ser amigable'*. Identifica las ambigüedades y sugiere preguntas para clarificarlas."

Salida Generada (IA)	**Ambigüedades detectadas:** 1. *'Rápida'*: No establece una métrica de tiempo aceptable. 2. *'Amigable'*: Es subjetivo y no verificable. **Preguntas sugeridas:** - ¿Cuál es el tiempo máximo en milisegundos aceptable para el procesamiento del pago? - ¿Qué estándar de accesibilidad o métrica de usabilidad (ej. clics máximos para completar la tarea) define 'amigable'?

Generación de Casos de Uso

A partir de requerimientos de alto nivel, la IA permite estructurar Casos de Uso formales y detallados, asegurando la cobertura de los flujos principales, los flujos alternativos (caminos secundarios) y los flujos de excepción (manejo de errores).

Elemento	Ejemplo Práctico
Prompt de Entrada	"Actúa como Analista de Sistemas. Genera un Caso de Uso estructurado para la funcionalidad 'Aplicar código de descuento en el carrito de compras'. Incluye: Nombre, Actor, Precondiciones, Flujo Principal y un Flujo de Excepción."

Salida Generada (IA)	**Nombre:** Aplicar Código de Descuento. **Actor:** Cliente Registrado. **Precondición:** El cliente tiene al menos un artículo en el carrito. **Flujo Principal:** 1. El cliente ingresa el código y selecciona 'Aplicar'. 2. El sistema valida la vigencia del código. 3. El sistema recalcula el total y muestra el descuento. **Flujo de Excepción (Código Inválido):** 2a. El sistema detecta que el código está caducado. 2b. El sistema muestra un mensaje de error: 'Código no válido o expirado' y mantiene el total original.

Integración del Marco Teórico

El rol del gestor de desarrollo de software requiere una transición operativa: pasar de la ejecución técnica directa a la planificación, supervisión y entrega estratégica. El conocimiento técnico previo es la base empírica que permite tomar decisiones fundamentadas, evaluar riesgos y estructurar el trabajo del equipo.

El Rol del Gestor de Software

El gestor actúa como el conector entre las necesidades del negocio y la viabilidad técnica. Sus funciones principales, sustentadas en su base técnica, son:

- Estimación objetiva: Evaluar tiempos y recursos necesarios comprendiendo la complejidad real del código, la arquitectura y los posibles bloqueos técnicos.
- Gestión de riesgos técnicos: Identificar cuellos de botella, acumulación de deuda técnica y limitaciones de infraestructura antes de que impacten el cronograma del proyecto.
- Traducción y comunicación: Convertir los requerimientos de negocio en especificaciones técnicas claras para los desarrolladores, y explicar las restricciones técnicas a los stakeholders en lenguaje de negocio o financiero.
- Aseguramiento de calidad (QA): Definir métricas de éxito, auditar estándares de código, garantizar la cobertura de pruebas y validar los criterios de aceptación.

Consolidación de Conocimientos Técnicos Previos

Los conocimientos adquiridos en materias anteriores cambian de propósito. El gestor no utiliza estas habilidades para construir el producto directamente, sino para administrar, auditar y optimizar su construcción.

Conocimiento Técnico Previo	**Aplicación Práctica en la Gestión**
Programación y Algoritmos	Revisión de decisiones de arquitectura, estimación de esfuerzo en iteraciones, identificación de deuda técnica.

Bases de Datos	Evaluación de escalabilidad del sistema, políticas de seguridad de la información, planificación de migraciones de datos.
Redes y Sistemas Operativos	Planificación de despliegues, evaluación de infraestructura en la nube y optimización de costos de servidor.
Análisis de Sistemas	Selección de metodologías de trabajo y diseño de procesos de elicitación de requerimientos adecuados para cada cliente.

Transición de Perfil: De Programador a Gestor

Asumir la gestión requiere un cambio de enfoque estructural para el técnico universitario:

- De la tarea al proceso: El enfoque prioritario pasa de escribir código funcional a garantizar que el flujo de trabajo del equipo (desde la definición del requerimiento hasta el despliegue) sea fluido y sin fricciones.

- Del individuo al equipo: La función principal es remover impedimentos (técnicos o burocráticos) para que el equipo pueda operar a máxima capacidad, en lugar de resolver el problema técnico de forma aislada.

- De la solución técnica al valor de negocio: Las decisiones tecnológicas (lenguajes, frameworks, arquitecturas) ya no se evalúan por preferencia técnica, sino en función de su retorno de inversión (ROI), costo de mantenimiento y alineación estricta con los objetivos del cliente.